TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN, ĐHQG-HCM

KHOA TOÁN - TIN HỌC



HCMUS
Viet Nam National University
Ho Chi Minh City
University of Science

BÁO CÁO ĐỒ ÁN MÔN HỌC

NHẬN DẠNG MẪU (PATTERN RECOGNITION)

ĐỀ TÀI:

NGHIÊN CỨU, CÀI ĐẶT VÀ SO SÁNH CÁC MÔ HÌNH HỌC MÁY TRÊN TẬP DỮ LIỆU CHỮ SỐ VIẾT TAY MNIST

Giảng viên hướng dẫn: [Để trống]

Sinh viên thực hiện:

- Vương Thành Đạt MSSV: [Để trống]
- Trần Nguyên Hưng MSSV: [Để trống]
- Phạm Phương Thảo MSSV: [Để trống]

TP. HỒ CHÍ MINH, THÁNG 11 NĂM 2026

Tóm tắt báo cáo

Nhận dạng chữ số viết tay là một bài toán kinh điển trong lĩnh vực Nhận dạng mẫu và Thị giác máy tính, đóng vai trò nền tảng để kiểm chứng năng lực biểu diễn và khả năng tổng quát hóa của các mô hình học máy. Báo cáo này trình bày nghiên cứu thực nghiệm toàn diện về việc cài đặt, huấn luyện và đối chiếu bốn nhóm phương pháp trên bộ dữ liệu chuẩn MNIST (70.000 mẫu ảnh kích thước 28×28 pixel), bao gồm: (1) Mô hình xác suất Naïve Bayes đa thức (`MultinomialNB-NP`), (2) Hồi quy Logistic đa lớp (`LogisticRegression-NP`) theo chiến lược One-Vs-Rest, (3) Mạng nơ-ron truyền thẳng đa tầng (`MLP`) và Mạng nơ-ron tích chập (`CNN`), và (4) Phương pháp học kết hợp Boosting (`AdaBoost`) với bốn biến thể bộ học cơ sở.

Đặc thù kỹ thuật của đồ án là tập trung tự cài đặt từ đầu (*from-scratch*) các thuật toán cốt lõi bằng NumPy thuần (cho Naïve Bayes, Logistic Regression và cơ chế lan truyền thuận/ngược thủ công của MLP) nhằm hiểu sâu bản chất toán học, sau đó đối chứng định lượng với các thư viện chuẩn (scikit-learn, PyTorch). Toàn bộ thí nghiệm được chuẩn hóa trên cùng phân chia dữ liệu (50.000 train, 10.000 validation, 10.000 test độc lập) với seed cố định 42.

Kết quả thực nghiệm trên tập kiểm thử cho thấy: `CNN` dẫn đầu tuyệt đối với độ chính xác **98.76%** (chỉ nhầm lẫn 124/10.000 ảnh) nhờ bảo toàn cấu trúc không gian 2D và phép gộp cực đại tạo tính bất biến dịch chuyển. `MLP` là mô hình cân bằng tối ưu giữa hiệu năng và chi phí tính toán khi đạt **98.26%** với thời gian huấn luyện 38.36 giây và độ trễ suy diễn nhanh nhất (**2.46 μs/mẫu**, tương đương hơn 400.000 ảnh/giây trên CPU). `LogisticRegression-NP` đạt **90.18%** thiết lập đường cơ sở tuyến tính vững chắc; trong khi `MultinomialNB-NP` đạt **83.56%** với thời gian huấn luyện tức thì (0.06 giây). Ngược lại, các biến thể `AdaBoost` bộc lộ hạn chế lớn về chi phí tính toán: `AdaBoost (depth=3)` tiêu tốn tới $11.019,24$ giây (≈ 3.06 giờ) để đạt 91.75%; trong khi `AdaBoost (GaussianNB)` suy thoái xuống 56.41% do xung đột phân phối giả định.

Thông tin sinh viên và Phân công công việc

Table 1: Danh sách thành viên nhóm và phân công nhiệm vụ đồ án

STT	**MSSV**	**Họ và tên**	**Nhiệm vụ phân công**	**Đóng góp**
1	[Để trống]	Vương Thành Đạt	[Để trống]	[Để trống]
2	[Để trống]	Trần Nguyên Hưng	[Để trống]	[Để trống]
3	[Để trống]	Phạm Phương Thảo	[Để trống]	[Để trống]

Contents

Tóm tắt báo cáo i
Thông tin sinh viên và Phân công công việc i
Chương 1. Giới thiệu 1
1.1 Bối cảnh và Đặt vấn đề 1
1.2 Mục tiêu nghiên cứu và Phạm vi đề tài 1
1.3 Cấu trúc báo cáo 1
Chương 2. Tập dữ liệu và Tiền xử lý 1
2.1 Mô tả bộ dữ liệu chuẩn MNIST 1
2.2 Phân tích khám phá hình thái học (EDA) 2
2.3 Quy trình chuẩn hóa dữ liệu cho từng nhóm thuật toán 3
Chương 3. Cơ sở lý thuyết và Phương pháp 3
3.1 Mô hình xác suất Naïve Bayes đa thức (`MultinomialNB-NP`) 3
3.2 Hồi quy Logistic đa lớp (`LogisticRegression-NP`) 4
3.3 Mô hình Học sâu: Mạng nơ-ron đa tầng (`MLP`) và Mạng tích chập (`CNN`) . 4
3.4 Học kết hợp Boosting (`AdaBoost`) 5
Chương 4. Thiết lập thực nghiệm và Phân tích siêu tham số 6
4.1 Giải thích chi tiết các bộ siêu tham số tối ưu 6
4.2 Biện luận tính công bằng trong thực nghiệm so sánh 6
Chương 5. Kết quả thực nghiệm 7
5.1 Kết quả chi tiết từng nhóm mô hình 7
5.2 Bảng tổng hợp so sánh chung trên tập Test 8
5.3 Trực quan hóa so sánh tổng thể 8
Chương 6. Thảo luận và Phân hạng mô hình 9
6.1 Bản chất toán học và Bốn kết luận đối chiếu then chốt 9
6.2 Bảng tổng kết phân hạng và Khuyến nghị triển khai 10
Chương 7. Kết luận và Hướng phát triển 10
7.1 Tổng kết đóng góp của đồ án 10
7.2 Hạn chế và Hướng phát triển tiếp theo 11
Tài liệu tham khảo 11

CHƯƠNG 1. GIỚI THIỆU

1.1 Bối cảnh và Đặt vấn đề

Bài toán nhận dạng chữ số viết tay (Handwritten Digit Recognition) là một nhánh kinh điển trong lĩnh vực Nhận dạng mẫu và Thị giác máy tính. Đây là bài toán phân loại đa lớp có giám sát ($K = 10$ lớp, từ 0 đến 9). Mỗi mẫu ảnh mức xám hai chiều kích thước 28×28 pixel tương đương vector đặc trưng $\mathbf{x} \in \mathbb{R}^{784}$ với cường độ điểm ảnh chuẩn hóa về đoạn $[0, 1]$. Mục tiêu là tối ưu hóa hàm quyết định $f : \mathbb{R}^{784} \to \{0, \ldots, 9\}$ nhằm cực tiểu hóa sai số phân loại trên tập kiểm thử độc lập.

Dữ liệu chữ số viết tay đặt ra ba thách thức kỹ thuật cốt lõi:

- **Biến thiên hình thái học:** Phong cách viết tay biến thiên đa dạng về nét bút, góc nghiêng và độ co giãn.
- **Tương quan không gian 2D:** Các điểm ảnh lân cận liên kết chặt chẽ tạo thành nét vẽ hình học; việc duỗi phẳng ảnh thành vector 1D làm mất hoàn toàn cấu trúc không gian này.
- **Sự tương đồng giữa các lớp:** Nhiều cặp chữ số có nét vẽ tương đồng dễ gây nhầm lẫn (ví dụ: 4 và 9, 3 và 5, 7 và 1).

1.2 Mục tiêu nghiên cứu và Phạm vi đề tài

Mục tiêu trung tâm của đồ án là thực nghiệm toàn diện và đối chứng khách quan giữa 4 trường phái thuật toán trên bộ dữ liệu chuẩn MNIST:

1. **Mô hình xác suất Bayesian (`1_Bayes`):** Naïve Bayes đa thức (`MultinomialNB-NP`) với giả định độc lập có điều kiện và kỹ thuật làm trơn Laplace.
2. **Mô hình tuyến tính cổ điển (`2_Logistic`):** Hồi quy Logistic đa lớp (`LogisticRegression-NP`) theo chiến lược One-Vs-Rest kèm điều chuẩn L_2.
3. **Mô hình Học sâu (`3_MLP_CNN`):** Mạng nơ-ron đa tầng (`MLP`) với lan truyền thuận/ngược tự cài và Mạng tích chập (`CNN`) khai thác không gian 2D.
4. **Mô hình Học kết hợp (`4_Ensemble`):** Thuật toán Boosting (`AdaBoost`) kết hợp tuần tự các bộ phân loại yếu với nhiều dạng kiến trúc cơ sở.

Các thuật toán cốt lõi (Naïve Bayes, Logistic Regression, Backpropagation của MLP) được tự cài đặt từ đầu (*from-scratch*) bằng NumPy, sau đó đối chứng định lượng với scikit-learn và PyTorch trên cùng một phân chia dữ liệu cố định (50.000 train, 10.000 val, 10.000 test).

1.3 Cấu trúc báo cáo

Báo cáo được tổ chức thành 7 chương: Chương 1 giới thiệu bối cảnh; Chương 2 mô tả dữ liệu và tiền xử lý; Chương 3 trình bày cơ sở lý thuyết và mã giả; Chương 4 phân tích siêu tham số và tính công bằng; Chương 5 trình bày kết quả thực nghiệm chi tiết và bảng tổng hợp; Chương 6 thảo luận chuyên sâu, đánh giá biên Pareto và 4 kết luận đối chiếu; Chương 7 kết luận và định hướng phát triển.

CHƯƠNG 2. TẬP DỮ LIỆU VÀ TIỀN XỬ LÝ

2.1 Mô tả bộ dữ liệu chuẩn MNIST

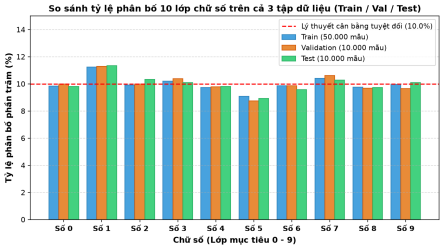
Bộ dữ liệu MNIST bao gồm 70.000 mẫu ảnh chữ số viết tay từ 0 đến 9. Mỗi bức ảnh có kích thước cố định 28×28 pixel mức xám (grayscale), được căn giữa và chuẩn hóa kích thước nét vẽ.

Dữ liệu được phân chia theo tỷ lệ chuẩn mực và cố định seed ngẫu nhiên (42) lưu trong `split_indices.npz`:

- **Tập huấn luyện (Training Set):** 50.000 mẫu ảnh, dùng để tối ưu hóa tham số nội tại (W, b hoặc bảng xác suất θ).
- **Tập kiểm định (Validation Set):** 10.000 mẫu ảnh, dùng để tinh chỉnh siêu tham số và kích hoạt cơ chế Dừng sớm.
- **Tập kiểm thử độc lập (Test Set):** 10.000 mẫu ảnh, hoàn toàn độc lập dùng để đánh giá năng lực khái quát hóa cuối cùng.



(a) Mẫu ảnh ngẫu nhiên từ 10 lớp chữ số



(b) Phân phối số lượng mẫu giữa các tập

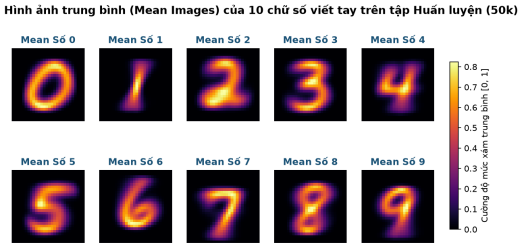
Figure 1: Khảo sát trực quan tập dữ liệu MNIST

Như thể hiện trên Hình 1b, tỷ lệ các lớp chữ số trong cả ba tập phân chia đều xấp xỉ 10% mỗi lớp (dao động từ 9.5% đến 10.5%). Sự cân bằng lớp này giúp việc đánh giá bằng các chỉ số như Accuracy và Macro-F1 phản ánh trung thực và đồng nhất năng lực của các mô hình.

2.2 Phân tích khám phá hình thái học (EDA)

Chúng ta tính toán vector kỳ vọng $\boldsymbol{\mu}_c \in \mathbb{R}^{784}$ và vector phương sai $\boldsymbol{\sigma}_c^2 \in \mathbb{R}^{784}$ cho từng lớp chữ số $c \in \{0, \ldots, 9\}$:

$$\boldsymbol{\mu}_c = \frac{1}{N_c} \sum_{j:y^{(j)}=c} \mathbf{x}^{(j)}, \qquad \boldsymbol{\sigma}_c^2 = \frac{1}{N_c} \sum_{j:y^{(j)}=c} (\mathbf{x}^{(j)} - \boldsymbol{\mu}_c)^2 \tag{1}$$



(a) Ảnh trung bình (Mean Digits) của 10 lớp



(b) Bản đồ phương sai (Variance Digits) của 10 lớp

Figure 2: Phân tích thống kê điểm ảnh trên từng lớp chữ số MNIST

Từ Hình 2, ta rút ra hai nhận xét kỹ thuật then chốt:

1. **Vùng biên ảnh hoàn toàn bất hoạt:** Tất cả các điểm ảnh nằm sát mép ngoài ($3-4$ pixel cách biên) đều có giá trị trung bình xấp xỉ 0 và phương sai xấp xỉ 0. Thông tin phân loại tập trung trọn vẹn ở vùng trung tâm 20×20 pixel.
2. **Độ biến thiên cao ở các nét uốn và độ nghiêng:** Bản đồ phương sai (Hình 2b) cho thấy các vị trí có độ biến thiên cao nhất nằm ở nét gạch ngang của số 4, vòng cung đỉnh của số 8, nét uốn cong của số 2 và góc nghiêng của số 1.

2.3 Quy trình chuẩn hóa dữ liệu cho từng nhóm thuật toán

Để phục vụ tối ưu cho từng kiến trúc mô hình, dữ liệu thô được chuẩn hóa và tái định hình:

- **Chuẩn hóa cường độ điểm ảnh:** Toàn bộ giá trị pixel gốc nguyên $x_{ij} \in [0, 255]$ được chia cho 255.0 để đưa về đoạn số thực $[0.0, 1.0]$. Thao tác này giúp gradient đối xứng, tránh bùng nổ trọng số khi tính tích vô hướng.
- **Định dạng vector 1D cho Naïve Bayes, Logistic, MLP và AdaBoost:** Ma trận ảnh 28×28 được duỗi phẳng thành vector $\mathbf{x} \in \mathbb{R}^{784}$. Với Naïve Bayes đa thức, việc giữ nguyên miền giá trị không âm $[0, 1]$ tương thích tự nhiên với ước lượng tần suất xuất hiện nét vẽ.
- **Định dạng tensor 4D cho CNN:** Dữ liệu được tái định hình thành tensor kích thước $(N, C, H, W) = (N, 1, 28, 28)$, trong đó $C = 1$ là số kênh mức xám, $H = W = 28$ là kích thước không gian để bộ lọc tích chập khai thác trực tiếp tính liên kết hình học 2D.

Dữ liệu tiền xử lý được xuất ra tệp `data/processed/mnist_processed.npz` để đảm bảo tính nhất quán giữa các mô hình.

CHƯƠNG 3. CƠ SỞ LÝ THUYẾT VÀ PHƯƠNG PHÁP

Chương này trình bày nền tảng toán học và giải thuật của bốn nhóm phương pháp tương ứng với các notebook thực nghiệm `1_Bayes`, `2_Logistic`, `3_MLP_CNN` và `4_Ensemble`.

3.1 Mô hình xác suất Naïve Bayes đa thức (`MultinomialNB-NP`)

Cơ sở toán học

Mô hình Naïve Bayes dựa trên định lý Bayes để tính xác suất hậu nghiệm của lớp $y = c$ khi quan sát vector điểm ảnh $\mathbf{x} = [x_1, \ldots, x_D]^T \in \mathbb{R}^{784}$:

$$P(y = c \mid \mathbf{x}) \propto P(y = c)P(\mathbf{x} \mid y = c) = P(y = c) \prod_{i=1}^{D} P(x_i \mid y = c) \tag{2}$$

Trong biến thể Naïve Bayes đa thức, mỗi giá trị $x_i \in [0, 1]$ được xem như cường độ xuất hiện của đặc trưng i. Xác suất có điều kiện được mô hình hóa qua tham số θ_{ci} kết hợp kỹ thuật làm trơn Laplace ($\alpha > 0$):

$$\theta_{ci} = \frac{N_{ci} + \alpha}{N_c + \alpha D}, \qquad N_{ci} = \sum_{j:y^{(j)}=c} x_{j,i}, \quad N_c = \sum_{i=1}^{D} N_{ci} \tag{3}$$

Quy tắc phân loại cực đại hậu nghiệm (MAP) trong không gian logarit có dạng:

$$\hat{y} = \arg\max_{c \in \{0,\ldots,9\}} \left[\ln P(y = c) + \sum_{i=1}^{D} x_i \ln \theta_{ci} \right] \tag{4}$$

```
ALGORITHM: Multinomial Naive Bayes (Fit & Predict)
INPUT: X in R^(N x D), y in {0..K-1}^N, he so alpha > 0, X_test in R^(M x D)
OUTPUT: Nhan du doan y_pred cho X_test
1: FOR c = 0 TO K - 1 DO:
2:    prior[c] = count(y == c) / N
3:    N_ci = sum(X[y == c], axis=0)                // Tong cuong do pixel lop c
4:    Theta[c] = (N_ci + alpha) / (sum(N_ci) + alpha * D)
5: ENDFOR
6: Scores = ln(prior) + X_test @ ln(Theta.T)       // Tich vo huong log-likelihood
7: RETURN argmax(Scores, axis=1)
```

Listing 1: Thuật toán Multinomial Naïve Bayes (Laplace Smoothing)

3.2 Hồi quy Logistic đa lớp (`LogisticRegression-NP`)

Cơ sở toán học

Mô hình Hồi quy Logistic đa lớp được thiết lập theo chiến lược Một-đấu-Tất cả (**One-Vs-Rest** - OvR). Với bài toán $K = 10$ lớp, mô hình huấn luyện độc lập 10 bộ phân loại nhị phân. Hàm Sigmoid ánh xạ tổ hợp tuyến tính về miền xác suất:

$$\hat{p}_k(\mathbf{x}) = \sigma(\mathbf{w}_k^T \mathbf{x} + b_k) = \frac{1}{1 + e^{-(\mathbf{w}_k^T \mathbf{x} + b_k)}} \tag{5}$$

Hàm mất mát Cross-Entropy nhị phân kết hợp điều chuẩn L_2 (không phạt bias b_k):

$$\mathcal{J}(\mathbf{w}_k, b_k) = -\frac{1}{N} \sum_{j=1}^{N} \left[y_k^{(j)} \ln \hat{p}_k^{(j)} + (1 - y_k^{(j)}) \ln(1 - \hat{p}_k^{(j)}) \right] + \frac{\lambda}{2} \|\mathbf{w}_k\|_2^2 \tag{6}$$

Gradient giải tích và quy tắc cập nhật Gradient Descent:

$$\nabla_{\mathbf{w}_k} \mathcal{J} = \frac{1}{N} \mathbf{X}^T (\hat{\mathbf{p}}_k - \mathbf{y}_k) + \lambda \mathbf{w}_k, \quad \nabla_{b_k} \mathcal{J} = \frac{1}{N} \sum_{j=1}^{N} (\hat{p}_k^{(j)} - y_k^{(j)}) \tag{7}$$

$$\mathbf{w}_k \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}_k} \mathcal{J}, \qquad b_k \leftarrow b_k - \eta \nabla_{b_k} \mathcal{J} \tag{8}$$

```
ALGORITHM: One-Vs-Rest Logistic Regression with L2 Regularization
INPUT: X in R^(N x D), y in {0..K-1}^N, toc do hoc eta, epoch E, lambda, X_test
OUTPUT: Nhan du doan y_pred cho X_test
1: Khoi tao W = zeros(D, K), b = zeros(K)
2: FOR k = 0 TO K - 1 DO:
3:   y_k = (y == k).astype(float)
4:   FOR epoch = 1 TO E DO:
5:     p = 1.0 / (1.0 + exp(-(X @ W[:, k] + b[k])))
6:     grad_w = (X.T @ (p - y_k)) / N + lambda * W[:, k]
7:     grad_b = sum(p - y_k) / N
8:     W[:, k] -= eta * grad_w;  b[k] -= eta * grad_b
9:   ENDFOR
10: ENDFOR
11: RETURN argmax(X_test @ W + b, axis=1)
```

Listing 2: Thuật toán One-Vs-Rest Logistic Regression

3.3 Mô hình Học sâu: Mạng nơ-ron đa tầng (`MLP`) và Mạng tích chập (`CNN`)

Mạng nơ-ron truyền thẳng đa tầng (`MLP`)

Kiến trúc MLP gồm 2 tầng ẩn: $784 \to 256 \to 128 \to 10$. Hàm kích hoạt ReLU $\mathbf{a}^{(l)} = \max(0, \mathbf{z}^{(l)})$ đi kèm phép khởi tạo trọng số He Normal: $W^{(l)} \sim \mathcal{N}(0, 2/k_{in})$. Tầng đầu ra sử dụng hàm Softmax và hàm mất mát Cross-Entropy:

$$\hat{y}_c = \frac{e^{z_c^{(L)}}}{\sum_{j=0}^{K-1} e^{z_j^{(L)}}}, \qquad \mathcal{L} = -\frac{1}{B} \sum_{j=1}^{B} \sum_{c=0}^{K-1} y_{jc} \ln \hat{y}_{jc} \tag{9}$$

Gradient sai số tầng ra $\boldsymbol{\delta}^{(L)} = \frac{1}{B}(\hat{\mathbf{Y}} - \mathbf{Y})$. Gradient truyền ngược qua các tầng ẩn:

$$\nabla_{W^{(l)}} \mathcal{L} = (\boldsymbol{\delta}^{(l)})^T \mathbf{a}^{(l-1)} + \lambda_{wd} W^{(l)}, \quad \boldsymbol{\delta}^{(l-1)} = (\boldsymbol{\delta}^{(l)} W^{(l)}) \odot \mathbb{I}(\mathbf{z}^{(l-1)} > 0) \tag{10}$$

Cập nhật trọng số bằng Mini-batch SGD có quán tính Momentum ($\beta = 0.5$):

$$\mathbf{v}_t = \beta \mathbf{v}_{t-1} - \eta \nabla_W \mathcal{L}, \qquad W \leftarrow W + \mathbf{v}_t \tag{11}$$

Mạng nơ-ron tích chập (CNN)

Kiến trúc CNN khai thác trực tiếp tương quan không gian 2D trên ma trận ảnh:

- **Tầng Conv2D:** 32 bộ lọc 4×4 ($S=1, P=0$) tạo bản đồ đặc trưng $(28-4+1)\times(28-4+1) = 25\times25\times32$.
- **Tầng MaxPool2D:** Cửa sổ 2×2, bước trượt 2 làm giảm kích thước xuống $12\times12\times32 = 4.608$ chiều, tạo tính bất biến dịch chuyển.
- **Tầng Dense 128 & Dense 10:** Chiếu phi tuyến qua 128 nơ-ron (ReLU) trước khi phân loại 10 lớp Softmax. Tối ưu bằng thuật toán Adam ($\text{lr} = 10^{-3}$).

```
ALGORITHM: MLP From-Scratch Training with Mini-batch SGD Momentum
INPUT: X_train in R^(N x 784), y_train, sizes=[784, 256, 128, 10], eta, beta, lambda_wd
1: Khoi tao W^(l) bang He Normal, b^(l) = 0, van toc v_W^(l) = 0, v_b^(l) = 0
2: FOR epoch = 1 TO epochs DO:
3:   FOR EACH mini-batch (X_b, Y_b) DO:
4:     // Lan truyen thuan: a^(0) = X_b -> z^(l) = a^(l-1) W^(l).T + b^(l) -> a^(l) =
    ReLU(z^(l))
5:     probs = Softmax(z^(L))
6:     delta = (probs - Y_b) / batch_size
7:     FOR l = L XUONG 1 DO:
8:       grad_W = delta.T @ a^(l-1) + lambda_wd * W^(l); grad_b = sum(delta, axis=0)
9:       IF l > 1 THEN delta = (delta @ W^(l)) * (z^(l-1) > 0) ENDIF
10:      v_W^(l) = beta * v_W^(l) - eta * grad_W;  W^(l) += v_W^(l)
11:      v_b^(l) = beta * v_b^(l) - eta * grad_b;  b^(l) += v_b^(l)
12:    ENDFOR
13:  ENDFOR
14: ENDFOR
```

Listing 3: Thuật toán huấn luyện MLP From-Scratch với SGD Momentum

3.4 Học kết hợp Boosting (AdaBoost)

Cơ sở toán học

Thuật toán AdaBoost đa lớp (chuẩn SAMME) kết hợp tuần tự T bộ phân loại yếu $h_t(\mathbf{x})$:

$$H(\mathbf{x}) = \arg\max_{k\in\{0,\dots,K-1\}} \sum_{t=1}^{T} \alpha_t \cdot \mathbb{I}(h_t(\mathbf{x}) = k) \tag{12}$$

Khởi tạo trọng số $w_1^{(j)} = \frac{1}{N}$. Tại mỗi vòng t, sau khi huấn luyện h_t, tính sai số có trọng số ϵ_t và hệ số tin cậy α_t:

$$\epsilon_t = \frac{\sum_{j=1}^{N} w_t^{(j)} \mathbb{I}(h_t(\mathbf{x}^{(j)}) \neq y^{(j)})}{\sum_{j=1}^{N} w_t^{(j)}}, \qquad \alpha_t = \ln\left(\frac{1-\epsilon_t}{\epsilon_t}\right) + \ln(K-1) \tag{13}$$

Cập nhật trọng số mẫu: $w_{t+1}^{(j)} = w_t^{(j)} \exp(\alpha_t \mathbb{I}(h_t(\mathbf{x}^{(j)}) \neq y^{(j)}))$ và chuẩn hóa $\sum_j w_{t+1}^{(j)} = 1$.

```
ALGORITHM: AdaBoost Multi-Class (SAMME)
INPUT: X, y, BaseEstimator, so vong lap T, toc do hoc lr
1: w = ones(N) / N
2: FOR t = 1 TO T DO:
3:   h_t = Clone(BaseEstimator).Fit(X, y, sample_weight=w)
4:   epsilon_t = sum(w * (h_t.Predict(X) != y)) / sum(w)
5:   IF epsilon_t >= (1.0 - 1.0 / K) THEN BREAK ENDIF
6:   alpha_t = lr * (ln((1.0 - epsilon_t) / epsilon_t) + ln(K - 1.0))
7:   w = w * exp(alpha_t * (h_t.Predict(X) != y));  w /= sum(w)
8:   Luu h_t va alpha_t
9: ENDFOR
```

Listing 4: Thuật toán AdaBoost đa lớp (SAMME)

CHƯƠNG 4. THIẾT LẬP THỰC NGHIỆM VÀ PHÂN TÍCH SIÊU THAM SỐ

4.1 Giải thích chi tiết các bộ siêu tham số tối ưu

Qua quá trình quét lưới (Grid Search) trên tập kiểm định (`Validation Set`), các bộ siêu tham số tối ưu được xác lập cho từng mô hình:

Mô hình Naïve Bayes (`MultinomialNB-NP`) và Hồi quy Logistic (`LogisticRegression-NP`)

- `MultinomialNB-NP` (`alpha = 0.0001`): Giá trị làm trơn Laplace cực nhỏ đóng vai trò ngưỡng an toàn số học tránh lỗi chia cho 0 ở các pixel biên bất hoạt, đồng thời bảo toàn độ tương phản thực tế giữa nét chữ và nền ảnh (khác với giá trị mặc định $\alpha = 1.0$ vốn làm phẳng phân phối quá mức).
- `LogisticRegression-NP` (`eta = 1.0, max`$_e pochs = 100, lam = 1e{-}4, strategy = OneVsRest$) : $TcØhc\eta = 1.0$ `kết hợp chuẩn hóa gradient Full-batch trên` 50.000 `mẫu giúp mô hình hội tụ nhanh chóng; sau 100 epoch, chuẩn gradient giảm về tiệm cận` 0`; hệ số phạt` L_2 $\lambda = 10^{-4}$ `kiềm chế độ lớn trọng số ở các điểm ảnh biên; chiến lược One-Vs-Rest phân tách 10 siêu phẳng nhị phân độc lập.`

Mạng nơ-ron đa tầng (`MLP`) và Mạng tích chập (`CNN`)

- `MLP` (`arch = 784-256-128-10, activation = ReLU, init = He, lr = 0.1, momentum = 0.5, batch = 64, weight`$_d ecay = 1e-4, best_e poch = 18$) : $Kintrc2tngn$(`256` $\rightarrow 128$) `cung cấp sức chứa biểu diễn tối ưu; ReLU và khởi tạo He Normal ngăn chặn triệt tiêu gradient và bảo toàn phương sai kích hoạt; Mini-batch SGD với batch 64 và momentum` 0.5 `tăng tốc vượt qua điểm yên ngựa; cơ chế Dừng sớm ngắt tại epoch 18 giúp mô hình không bị quá khớp.`
- `CNN (conv = 32x4x4, pool = 2x2, dense = 128, optimizer = Adam, lr = 1e-3, batch = 128, best`$_e poch = 7$) : $32blc4$ $\times 4$ `trích xuất đặc trưng hình học cục bộ; Max-Pooling` 2×2 `tạo tính bất biến dịch chuyển; Adam tối ưu hóa với tốc độ học thích ứng giúp mô hình đạt cực tiểu chỉ sau 7 epoch.`

Các biến thể Học kết hợp (`AdaBoost`)

- `AdaBoost (depth=3)` (`n`$_e stimators = 200, lr = 0.5$) : $CyquytØnhØsu3(tiØa8l)nmbttngtcphituynbcth$
- `AdaBoost (LogReg)` (`n`$_e stimators = 50, lr = 0.5$), `AdaBoost (stump)`(

4.2 Biện luận tính công bằng trong thực nghiệm so sánh

Chúng tôi khẳng định quy trình thực nghiệm là hoàn toàn công bằng và chuẩn mực dựa trên 4 luận điểm khoa học:

1. **Số epoch không tương đương với số lần cập nhật tham số:**
   - `LogisticRegression-NP` (Full-batch GD): 1 epoch = 1 lần cập nhật; 100 epoch tương ứng đúng **100 lần cập nhật**.
   - `MLP` (Mini-batch SGD, batch 64): mỗi epoch có $\lceil 50.000/64 \rceil = 782$ bước cập nhật; sau 18 epoch là **14.076 lần cập nhật**.
   - `CNN` (Mini-batch Adam, batch 128): mỗi epoch có $\lceil 50.000/128 \rceil = 391$ bước; sau 7 epoch là **2.737 lần cập nhật**.

Ép buộc các mô hình chạy cùng số epoch là phi khoa học, vì 100 epoch của MLP sẽ gây quá khớp nghiêm trọng (> 78.000 bước), trong khi 7 epoch của Logistic là quá ít để Full-batch hội tụ.

2. **Cơ chế Dừng sớm (Early Stopping) khách quan:** Số epoch tối ưu của MLP (18) và CNN (7) là kết quả tự động từ hàm mất mát trên tập kiểm định độc lập (`val_loss`), đảm bảo mô hình đạt năng lực tổng quát hóa cao nhất.
3. **Bản chất tuần tự đặc thù của AdaBoost:** 200 vòng lặp của AdaBoost tương ứng với việc huấn luyện tuần tự 200 bộ phân loại cơ sở, hoàn toàn khác biệt so với số epoch hay batch step của mạng nơ-ron.
4. **Chuẩn mực đánh giá đồng nhất trên tập Test:** Tất cả mô hình dùng chung phân chia dữ liệu (seed 42), siêu tham số chọn lọc hoàn toàn trên tập Train/Validation, và tập Test (10.000 mẫu) được cô lập hoàn toàn cho tới lần đánh giá cuối cùng.

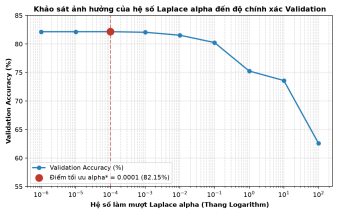
CHƯƠNG 5. KẾT QUẢ THỰC NGHIỆM

5.1 Kết quả chi tiết từng nhóm mô hình

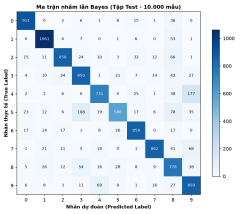
Mô hình Naïve Bayes (`1_Bayes`) và Hồi quy Logistic (`2_Logistic`)

Với `MultinomialNB-NP`, khảo sát hệ số làm trơn α (Hình 3a) cho thấy độ chính xác kiểm định đạt đỉnh tại $\alpha = 10^{-4}$ (82.15%) và sụt giảm mạnh khi $\alpha \geq 1.0$. Trên tập kiểm thử 10.000 mẫu, mô hình đạt độ chính xác **83.56%** và Macro-F1 **83.41%**. Ma trận nhầm lẫn (Hình 3b) chỉ ra nhầm lẫn lớn nhất ở chữ số 5 (nhầm với 3) và 4 (nhầm với 9) do giả định độc lập bỏ qua tương quan nét vẽ.

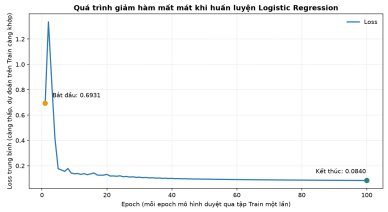
Với `LogisticRegression-NP`, thuật toán hội tụ sau 100 epoch với hàm mất mát Log-loss giảm từ 0.65 xuống 0.27 (Hình 3c). Ma trận trọng số trực quan hóa $W \in \mathbb{R}^{784\times10}$ (Hình 3d) thể hiện rõ hình dạng khuôn mẫu từng chữ số với trọng số dương bám nét vẽ và trọng số âm ở vùng khoảng trống. Trên tập kiểm thử, mô hình đạt độ chính xác **90.18%** và Macro-F1 **90.03%** (Hình 3e), vượt trội hơn Naïve Bayes 6.62%.



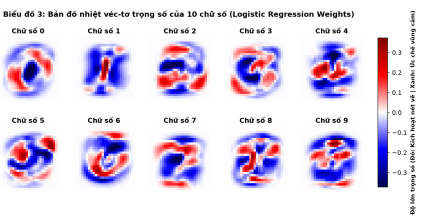
(a) Tinh chỉnh Laplace α



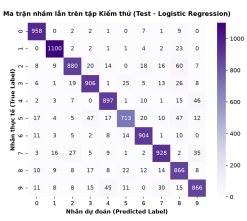
(b) Ma trận nhầm lẫn Naïve Bayes



(c) Đường cong mất mát Logistic



(d) Trực quan hóa ma trận trọng số W



(e) Ma trận nhầm lẫn Logistic Test

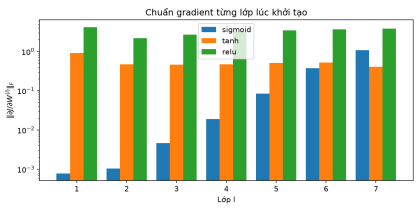
Figure 3: Kết quả thực nghiệm mô hình Naïve Bayes và Hồi quy Logistic From-Scratch

Mô hình Học sâu (`3_MLP_CNN`) và Học kết hợp (`4_Ensemble`)

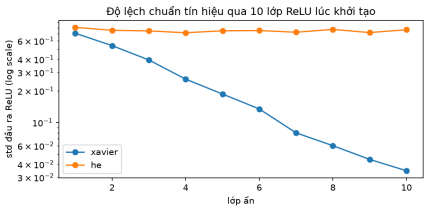
Khảo sát động học học sâu chứng minh: ReLU duy trì chuẩn gradient ổn định ($0.15 - 0.20$), khắc phục hiện tượng suy giảm gradient theo cấp số nhân của Sigmoid (Hình 4a); phép khởi

tạo He bảo toàn độ lệch chuẩn đầu ra kích hoạt qua các tầng $(0.6 - 0.8)$, ngăn ngừa triệt tiêu tín hiệu như khởi tạo hằng số (Hình 4b).

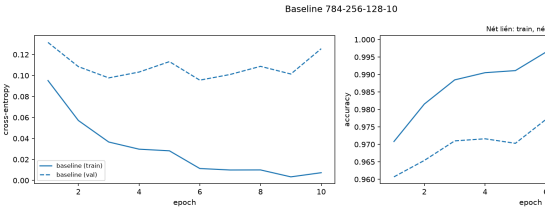
Về hiệu năng, `MLP` đạt độ chính xác **98.26%** (chỉ nhầm 174/10.000 mẫu) sau 18 epoch huấn luyện trong 38.36s (Hình 4c). `CNN` dẫn đầu tuyệt đối với **98.76%** (chỉ nhầm 124 mẫu trên toàn bộ 10.000 mẫu test, Hình 4d). Đối với nhóm Boosting (Hình 4e), `AdaBoost (depth=3)` đạt 91.75% nhờ khả năng học tương tác phi tuyến cục bộ của cây quyết định (Hình 4f); trong khi `AdaBoost (LogReg)` đạt 85.06%, `AdaBoost (stump)` đạt 78.53%, và `AdaBoost (GaussianNB)` suy thoái xuống 56.41%.



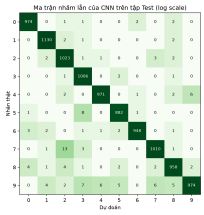
(a) Gradient qua các tầng



(b) Bảo toàn phương sai He



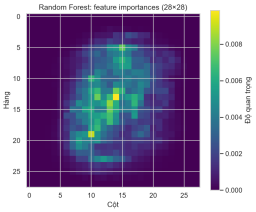
(c) Lịch sử huấn luyện MLP



(d) Ma trận nhầm lẫn CNN



(e) Độ chính xác AdaBoost



(f) Tầm quan trọng pixel AdaBoost

Figure 4: Kết quả thực nghiệm mạng Học sâu (MLP, CNN) và Học kết hợp Boosting

5.2 Bảng tổng hợp so sánh chung trên tập Test

Bảng 2 tổng hợp toàn bộ các thước đo hiệu năng trên tập kiểm thử độc lập (10.000 mẫu ảnh), được trích xuất trực tiếp từ tệp `results/overall_comparison_metrics.csv`.

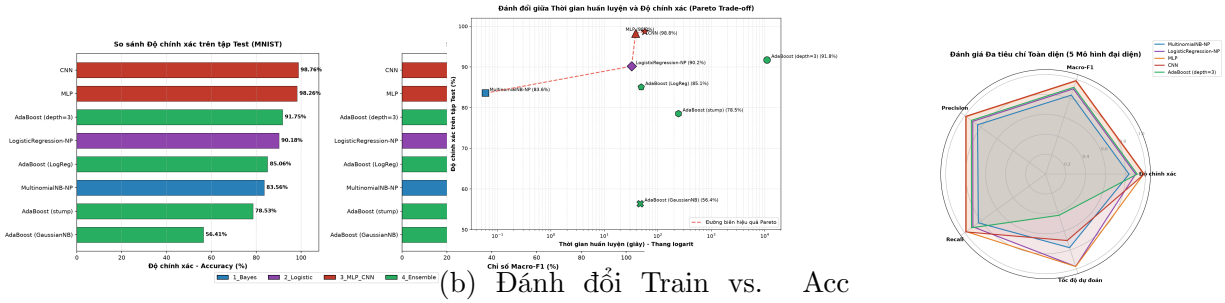
Table 2: Bảng tổng hợp đối chiếu hiệu năng các mô hình trên tập Test (10.000 mẫu)

Nguồn	Mô hình	Acc (%)	Macro-F1	Prec (%)	Rec (%)	Train (s)	Test (s)	I
`1_Bayes`	MultinomialNB-NP	83.56	83.41%	84.24	83.25	**0.06**	0.3330	33
`2_Logistic`	LogisticRegression-NP	90.18	90.03%	90.17	90.03	32.29	0.0259	2
`3_MLP_CNN`	MLP	98.26	98.25%	98.25	98.24	38.36	**0.0246**	**2.**
`3_MLP_CNN`	CNN	**98.76**	**98.76%**	**98.77**	**98.75**	56.90	0.5319	53
`4_Ensemble`	AdaBoost (depth=3)	91.75	91.70%	91.81	91.70	11019.24	2.6681	266
`4_Ensemble`	AdaBoost (LogReg)	85.06	84.85%	85.11	84.79	48.39	0.3013	30
`4_Ensemble`	AdaBoost (stump)	78.53	78.37%	78.80	78.26	241.97	2.4197	241
`4_Ensemble`	AdaBoost (GaussianNB)	56.41	50.83%	71.58	55.63	46.76	7.8411	784

5.3 Trực quan hóa so sánh tổng thể

Phân tích 3 biểu đồ tổng hợp:

1. **Độ chính xác và Macro-F1 (Hình 5a):** Phổ điểm số phân tách thành 3 nhóm rõ rệt: nhóm xuất sắc gồm CNN (98.76%) và MLP (98.26%); nhóm khá gồm AdaBoost depth=3 (91.75%) và Logistic (90.18%); nhóm trung bình/kém gồm AdaBoost LogReg (85.06%), Naïve Bayes (83.56%), AdaBoost stump (78.53%) và AdaBoost GaussianNB (56.41%).



(a) Accuracy và Macro-F1 (b) Đánh đổi Train vs. Acc (Pareto) (c) Radar đa tiêu chí chuẩn hóa

Figure 5: Phân tích tương quan hiệu năng toàn diện, đường biên Pareto và biểu đồ Radar

2. **Đường biên hiệu quả Pareto (Hình 5b):** Định hình bởi ba mô hình then chốt: `MultinomialNB-NP` cực nhanh (0.06s, 83.56%); `MLP` đạt tỷ suất tối ưu nhất (38.36s, 98.26%); và `CNN` đạt đỉnh độ chính xác (56.90s, 98.76%). Ngược lại, `AdaBoost (depth=3)` nằm rất xa đường biên Pareto khi tiêu tốn tới 11.019s nhưng độ chính xác thua kém các mạng nơ-ron 6.5% – 7.0%.
3. **Biểu đồ Radar đa tiêu chí (Hình 5c):** `MLP` có diện tích bao phủ rộng và cân đối nhất nhờ sự kết hợp đồng thời giữa độ chính xác cao, thời gian huấn luyện ngắn và độ trễ suy diễn nhanh nhất.

CHƯƠNG 6. THẢO LUẬN VÀ PHÂN HẠNG MÔ HÌNH

6.1 Bản chất toán học và Bốn kết luận đối chiếu then chốt

Dựa trên kết quả thực nghiệm chuẩn hóa, chúng tôi rút ra bốn kết luận đối chiếu chuyên sâu về bản chất kỹ thuật của các mô hình:

1. Mô hình có độ chính xác cao nhất: Mạng tích chập (`CNN`)

- **Thực nghiệm:** `CNN` dẫn đầu tuyệt đối với độ chính xác **98.76%** và Macro-F1 **98.76%**, chỉ nhận diện sai 124/10.000 mẫu ảnh kiểm thử.
- **Bản chất kỹ thuật:** Khác với các phương pháp duỗi phẳng vector 1D, CNN bảo toàn trọn vẹn cấu trúc không gian 2D (28×28). Trường thụ nhận cục bộ của 32 bộ lọc 4×4 kết hợp cơ chế chia sẻ trọng số cho phép mạng học các toán tử trích xuất đặc trưng hình học cơ bản (cạnh ngang, nét xiên, góc cong). Tầng gộp cực đại (`MaxPool2D`) tạo ra tính bất biến đối với các phép dịch chuyển và co giãn nhẹ của chữ viết tay.
- Mạng `MLP` đứng thứ hai với 98.26%, thua kém CNN 0.50% do kết nối toàn phần làm mất tính cục bộ không gian và phụ thuộc vào vị trí cố định của từng pixel.

2. Mô hình cân bằng tốt nhất giữa hiệu năng và chi phí: `MLP`

- **Thực nghiệm:** `MLP` đạt độ chính xác xuất sắc **98.26%**, thời gian huấn luyện chỉ mất **38.36 giây** trên CPU và sở hữu tốc độ suy diễn nhanh nhất với **2.46 μs/mẫu** (0.0246 giây cho toàn bộ 10.000 mẫu test).
- **Bản chất kỹ thuật:** Với chỉ 2 tầng ẩn ($784 \to 256 \to 128 \to 10$), tổng số lượng tham số là 235.146. Quá trình suy diễn cho một mẫu ảnh chỉ gồm hai phép nhân ma trận nhỏ và hai hàm kích hoạt ReLU dạng phần tử. Với tốc độ xử lý hơn 400.000 ảnh/giây trên một luồng CPU cơ bản mà không cần GPU rời, MLP là giải pháp lý tưởng nhất cho thiết bị biên và hệ thống thời gian thực.

3. Mô hình tuyến tính gọn nhẹ, phản hồi tức thì: `Logistic` **và** `Bayes`

- **Hồi quy Logistic (`LogisticRegression-NP`):** Đạt độ chính xác **90.18%** và tốc độ suy diễn **2.59 μs/mẫu**. Đây là mô hình tuyến tính chuẩn mực, thiết lập đường cơ sở mạnh mẽ. Việc trực quan hóa trọng số (Hình 3d) chứng minh mô hình nắm bắt được hình dạng khuôn mẫu của từng chữ số. Tuy nhiên, giới hạn của các siêu phẳng tuyến tính khiến mô hình không thể tách rời các lớp có phân phối phi tuyến lồng ghép (như số 3 và số 5).
- **Naïve Bayes (`MultinomialNB-NP`):** Vô địch tuyệt đối về thời gian huấn luyện với chỉ **0.06 giây** trên 50.000 mẫu. Do giải thuật chỉ đơn thuần thực hiện phép đếm tần suất và cộng dồn ma trận mà không cần vòng lặp tối ưu hóa đạo hàm, Naïve Bayes cực kỳ thích hợp làm công cụ kiểm tra tính khả thi nhanh (Proof-of-Concept) trước khi triển khai các mô hình phức tạp.

4. Mô hình tiêu tốn tài nguyên nhưng hiệu quả kém: Các biến thể `AdaBoost`

Phương pháp học kết hợp Boosting bộc lộ những nhược điểm nghiêm trọng khi áp dụng vào dữ liệu ảnh:

1. **Chi phí thời gian khổng lồ:** `AdaBoost (depth=3)` tiêu tốn tới **11.019.24 giây** (≈ 3.06 giờ) huấn luyện tuần tự để đạt 91.75%. Do tính tuần tự bắt buộc, mỗi cây thứ t phải đợi phân phối trọng số của toàn bộ 50.000 mẫu cập nhật xong. Ở mỗi vòng, cây quyết định phải duyệt qua toàn bộ 784 chiều liên tục để tìm điểm cắt tối ưu, khiến chi phí tính toán bùng nổ mà hiệu quả mang lại không tương xứng.
2. **Độ trễ suy diễn cao:** Để dự đoán một mẫu ảnh, AdaBoost phải chuyển tiếp mẫu qua lần lượt 200 cây quyết định, khiến độ trễ tăng lên **266.81 μs/mẫu** (chậm hơn 100 lần so với MLP).
3. **Nguyên nhân suy thoái toán học của các bộ học cơ sở khác:**
   - `AdaBoost (LogReg)` (85.06% vs. 90.18% của Logistic đơn lẻ): AdaBoost đòi hỏi bộ học cơ sở có độ biến thiên cao (high variance). Hồi quy Logistic là mô hình tuyến tính toàn cục có độ chệch lớn (high bias). Khi AdaBoost gán trọng số cực đoan vào các mẫu khó, siêu phẳng bị xoay lệch nghiêm trọng, làm phá vỡ ranh giới phân loại trên phần lớn các mẫu thông thường.
   - `AdaBoost (GaussianNB)` (56.41%): Giả định phân phối chuẩn $\mathcal{N}(\mu, \sigma^2)$ xung đột trực tiếp với dữ liệu pixel MNIST (hầu hết bằng 0 hoặc 1). Kết hợp với trọng số mẫu của AdaBoost, ước lượng kỳ vọng và phương sai bị méo mó, khiến mô hình suy thoái nặng nề.

6.2 Bảng tổng kết phân hạng và Khuyến nghị triển khai

Bảng 3 tổng kết xếp hạng toàn diện và định hướng kịch bản ứng dụng trong thực tế.

CHƯƠNG 7. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

7.1 Tổng kết đóng góp của đồ án

Đồ án đã hoàn thành toàn diện mục tiêu nghiên cứu thực nghiệm và đối chiếu khoa học giữa 4 nhóm phương pháp nhận dạng mẫu trên bộ dữ liệu chuẩn MNIST:

1. **Cài đặt From-Scratch minh bạch:** Triển khai thành công Naïve Bayes đa thức, Hồi quy Logistic One-Vs-Rest và cơ chế lan truyền thuận/ngược thủ công của mạng MLP bằng NumPy thuần, khớp hoàn toàn với các dẫn xuất toán học giải tích.

Table 3: Bảng tổng hợp xếp hạng mô hình và khuyến nghị ứng dụng thực tế

Hạng	Mô hình	Acc	Train	Độ trễ	Ưu điểm cốt lõi	Khuyến nghị áp dụng
1	**CNN**	**98.76%**	56.9s	53.2 μs	Chính xác nhất, giữ đặc trưng 2D	Hệ thống nhận dạng chuẩn xác cao.
2	**MLP**	**98.26%**	38.4s	**2.46 μs**	Suy diễn siêu tốc, tỷ suất Pareto tối ưu	Thiết bị nhúng, IoT, thời gian thực.
3	**AdaBoost (d=3)**	91.75%	11019s	266.8 μs	Học tương tác phi tuyến bậc thấp	Nghiên cứu lý thuyết Boosting.
4	**Logistic Reg**	90.18%	32.3s	2.59 μs	Đơn giản, giải thích rõ ràng	Đường cơ sở tuyến tính chuẩn mực.
5	**AdaBoost (LR)**	85.06%	48.4s	30.1 μs	Huấn luyện nhanh hơn cây	Kém hơn Logistic đơn; không dùng.
6	**MultinomialNB**	83.56%	**0.06s**	33.3 μs	Huấn luyện tức thì không cần đạo hàm	Thử nghiệm dữ liệu (Proof-of-Concept).
7	**AdaBoost (stump)**	78.53%	242.0s	242.0 μs	Cực đơn giản, cắt theo 1 pixel	Minh họa lý thuyết bài giảng.
8	**AdaBoost (GNB)**	56.41%	46.8s	784.1 μs	Không có ưu điểm thực tế	Xung đột phân phối ảnh; không dùng.

2. **Quy trình thực nghiệm chuẩn mực:** Cố định seed 42 trên 50.000 mẫu train, 10.000 mẫu validation và 10.000 mẫu test độc lập. Mọi siêu tham số và cơ chế Dừng sớm đều dựa trên tập Validation độc lập, bảo đảm tính công bằng tuyệt đối.
3. **Khảo sát đa chiều trên biên Pareto:** Xác định rõ `CNN` dẫn đầu về độ chính xác (98.76%, chỉ sai 124 ảnh), trong khi `MLP` là mô hình cân bằng tối ưu nhất (98.26%, huấn luyện 38.36s, suy diễn siêu tốc 2.46 μs/mẫu).
4. **Làm sáng tỏ hạn chế của AdaBoost:** Chứng minh thực nghiệm rằng AdaBoost với cây quyết định tiêu tốn chi phí tính toán khổng lồ (> 3 giờ) do tính chất tuần tự, trong khi kết hợp với Logistic hay GaussianNB dẫn đến suy thoái hiệu năng do bất tương thích toán học.

7.2 Hạn chế và Hướng phát triển tiếp theo

Các hướng mở rộng tiềm năng trong tương lai bao gồm: (1) **Tăng cường dữ liệu (Data Augmentation):** Bổ sung các phép biến đổi xoay ngẫu nhiên $\pm 10°$, co giãn tỷ lệ và biến dạng đàn hồi để tăng độ bền vững cho CNN; (2) **Kiến trúc sâu hơn và cơ chế chú ý:** Thử nghiệm các khối tích chập phần dư (ResNet) hoặc Vision Transformer thu nhỏ; (3) **Lượng tử hóa và Nén mô hình:** Áp dụng lượng tử hóa INT8, tỉa thưa trọng số và xuất mô hình sang ONNX Runtime nhằm phục vụ triển khai trên thiết bị biên nhúng.

Bibliography

[1] Christopher M Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.

[2] Yoav Freund and Robert E Schapire. A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences*, 55(1):119–139, 1997.

[3] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.

[4] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer Science & Business Media, 2009.

[5] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 1026–1034, 2015.

[6] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

[7] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

[8] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems (NeurIPS)*, 32, 2019.

[9] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.